

Using Machine Learning techniques to analyze Impact of Automobile Features on Testing Time in an Automotive Manufacturing setting

Kesav Kalanidhi

December 9th, 2025

I Introduction

Cars are incredibly complex and thus require thorough testing before they are ready to be sold. For this reason, modern-day automobile assembly plants include efficient testing systems for cars straight off of the assembly line, capable of completing testing procedures in mere minutes. However, the time required for a car to pass testing depends largely on the features that the car has. While some features may necessitate extra testing time when present, others may not have an impact on testing time, and it is also possible that different features can have similar impacts.

The MBGM (Mercedes-Benz Greener Manufacturing) dataset^[1] has categorical and binary data on the presence and absence of 377 different features in over four thousand automobiles. All of the dataset's features represent actual features in Mercedes-Benz cars. The target variable in this dataset is testing time. All of the data is unlabeled, meaning that the features are anonymized and that there is no way to tell which data feature corresponds to a certain car feature.

The problem that this dataset points to is predicting testing time based on the features that a car has. The biggest challenge this problem presents is the presence of so many features, making it hard to visualize the data and even harder to determine which features are relevant to the target variable from a predictive perspective. Additionally, some of the features are categorical and cannot be numerically analyzed. However, several machine learning techniques including preprocessing, dimensionality reduction, feature selection, regularization, gradient descent, and stochastic learning can be utilized to find an optimal prediction model that takes in features and outputs a predicted testing time.

While very conspicuous, this problem presents a relatively obscure opportunity to make inferences about the features. One potential inference is causation, such as predicting which features increase testing time the most or which features have no discernible effect on testing time. Another possible inference is feature covariance, or the extent to which two features have similar impacts on testing time.

II Goals

This project contains two major goals in which machine learning techniques will be used to investigate an underlying problem and model analysis will be used to make conclusions.

The first goal of this project is to build a model capable of taking a list of cars with all 377 features and giving an accurate testing time prediction for each car. The second goal is being able

to use the conclusions from the first goal to determine significant relationships both between two features and from a feature to the target.

III Model Research

A wide range of basic models, using a variety of techniques and serving different purposes, were all selected to conduct further research^[2] on as potential candidates for the prediction model. Research efforts were centered around determining if the purpose of each model aligned with the problem at hand and learning about the pros and cons of each model.

By nature, predicting a certain value is a regression problem. Thus, the first models to be further investigated were linear regression models. The plain linear model consisted of simple least-squares regression, with the sole purpose of minimizing the MSE (mean squared error) of the predictions. This model is often a useful baseline to determine the target variable's overall predictability, but it tends to be quite sensitive to features when there are many of them. Variants of the linear model included L1 and L2 regularization, both of which penalize weight values based on their size. This penalty is incorporated into the loss function that linear models strive to minimize through optimal weight choices. L1 regularization penalizes non-zero weights according to their value and is against weights of any magnitude, often driving weights to zero. L2 regularization penalizes non-zero weights according to the square of their value and is against weights of large magnitudes, often driving weights to small non-zero values. Lasso regression uses an L1 penalty, Ridge regression uses an L2 penalty, and Elastic net regression employs both. While L1 penalties promote feature selection by dropping several, L2 penalties account for different features with similar impacts on the target variable. As a result of this, Lasso regression

tends to perform well on sparse datasets while Ridge regression favors high-dimensional data with lots of inter-feature collinearity. Elastic-net regression is a balance of Lasso and Ridge models but is more computationally expensive and difficult to train. As the MBGM dataset is high-dimensional yet relatively sparse, all of the above models are worth trying.

The next group of models worth attention is those that center around dimensionality reduction and use different strategies to compress or filter high-dimensional data into a smaller set of features. Many of these models depend on feature variance in some way. The classic example of this is PCA (principal component analysis), which compresses data into new features based on how much overall variance in the data they capture. This makes it a useful baseline to consider before trying other models. There are many other feature selection techniques with relatively low computation such as variance thresholds or k-best selectors, which purely rely on ranking features by their variance. Admittedly, these strategies suffer from not considering collinearity between variables. In addition to this, PCA and other feature selection methods are strictly *tools* and not *models* - they provide no predictive inference, but rather serve to help with visualizing complex data, understanding patterns, and simplifying high-dimensional data by “cutting out the clutter”. In order to transform data and then rely on standard regression methods to give predictions, these strategies rely on the assumption that the most variant features are the most useful ones in predictive tasks.

There is a variance-based approach to regression that is equipped to handle high-dimensional data with lots of potential collinearity. This approach is PLS (partial least squares) regression, which centers around the concept of covariance. Essentially, PLS keeps and uses the features that

have the most variance in common with the target variables. The only drawback of this method is its narrow approach - only features with maximum variance relative to the target data will be considered, even if there are less variant features that have the potential to explain more of the variance within the target dataset.

One issue with model training that this dataset presents is its combination of high sample count and high feature count, making any operations involving the full dataset computationally costly. Thus, stochastic models that train on data in batches are of use in this case. One of these models uses a spin on gradient descent called SGD (stochastic gradient descent) in which small batches of data are used at each step to iteratively get closer to an optimal weight combination. This model is SGD regression, but it is only efficient with enough data and has lots of hyperparameters. A neural network has a more customizable architecture and is better at modeling complex relationships, but is the most computationally intensive and difficult to train model.

One important aspect of the data is the fact that each feature has a finite number of states. Thus, another family of models that could prove useful is the group of models reliant on decision trees. A simple decision tree model would promote simplicity, easy visualization, and easy training. However, one tree can easily overfit during training, unlike a random forest of many trees randomly assigned to different batches of data. However, a random forest model possesses many hyperparameters and has lots of computation involved.

There were plenty of models that were originally looked into that were not deemed useful for the purposes of this project. A majority of the rejected models were geared towards classification problems rather than regression problems, and favored strategies that created hard boundaries between classes rather than continuous spectrums of change in target variables. These included Linear Discriminant Analysis and Support Vector Machines.

IV Methods

The first step in model development was to load and preprocess the dataset. Pandas was used to load the data into an array structure and convert the categorical variables to numerical data using one-hot encoding. Then, the feature data and the target variables were both extracted into two separate arrays. Beyond this, no other preparation was needed, which is a benefit of using an existing dataset.

A standardized procedure for model analysis was coded prior to training and testing. The procedure began with ten-fold cross-validation and used the predictions from all ten validation folds to calculate and return the MSE of the model. Because the problem was regression-based, MSE was seen as the best metric to evaluate model performance. All models tested in the development phase except for the neural network were sourced from the scikit-learn package.

The training process for all models except for the neural network started with trying the default version of the model. Then, further research was conducted on each model to determine which hyperparameters controlled which aspects of the model and which ones should be tuned. Then, tuning consisted of using trial and error on one hyperparameter at a time until the optimal value

was found. The trial and error attempts are commented out in the development code to showcase the tuning process.

Training the neural network was significantly more difficult and challenging due to the need to construct a new class, the use of PyTorch to create a neural network with customizable architecture, and the computationally expensive and inconsistent model analyses. Countless different network architectures were tried until the architecture could not be improved upon by the cycle involving trial and error as well as extensive research^[3] on layers. For simplicity, only the final network architecture is shown in the development code, although the decisions made with respect to layers will be elaborated on later. The optimizer algorithm was also chosen carefully from expansive research. In addition to finding an optimal architecture and optimizer, many of the neural network's hyperparameters such as learning rate, batch size, and number of epochs required individual tuning to improve model performance further.

V Model Development

Before any models were developed, the data was visualized through basic 2-component PCA. It was found that the relationship between one of the PCA components and the target variable had an extremely high correlation, but when this component was passed into a plain Linear Regression model, the MSE was the exact same as the MSE for Linear Regression on the feature data. No tuning was required for this model.

Next, regularization penalties were introduced through the Lasso, Ridge, and Elastic Net models. For all of these models, the total penalty magnitude required tuning. For the Elastic Net, the L1

to L2 ratio was tuned until it was found that a full L1 penalty was favored. This was consistent with the results of the L1 and L2 models, because after effectively tuning both models, the L1 model had a lower MSE than the L2 model. Thus, to test the effectiveness of the feature selection data transformers, the L1 model with an optimal penalty coefficient of 0.025 was used.

Two different variance-based transformers were used to investigate the effectiveness of feature selection. In this case, since there are only two possible values for each feature, variance is essentially the evenness of the two values of a feature amongst all the samples. The first transformer used a variance threshold to remove all features without a variance that was at least a certain value. The second transformer used a k-best selector to keep only a certain amount of features with the highest variances. The variance threshold and the feature count were the hyperparameters that were tuned for their respective models. When evaluated using the optimal L1 model discussed above, both transformers yielded good results, with the variance threshold being able to slightly improve upon the MSE of the optimal L1 model.

The intersection between feature variance and dimensionality reduction was examined even more closely with the development of the PLS Regression model. The only hyperparameter to tune was the number of components to keep, but the PLS model did worse than other models when evaluated.

Next, stochastic techniques were applied through batch training with the SGD Regressor. While tuning the initial learning rate was always necessary, two different learning rate strategies were tried. The first strategy was keeping the learning rate constant while the second strategy was

scaling down the learning rate with each iteration. Both strategies yielded similar MSEs after finding the optimal initial learning rate and were roughly as good as the PLS model.

Next, the neural network was developed. Because a customizable architecture was desired, PyTorch was used to source the layers, optimizer, and propagation functions. However, the standardized model analysis procedure required that the model have the fit and predict methods that the scikit-learn models have built into them, and PyTorch did not support these methods. The solution was to create a hybrid class between a scikit-learn estimator and a PyTorch neural network, which was branded a neural network estimator. The fit method zeroed gradients, made a forward pass, calculated loss, made a backward pass, and updated weights for each batch in each epoch. The predict method used the fitted model with weights to calculate predictions for each sample. Hyperparameters to tune included learning rate, batch size, and epoch count. The SGD optimizer with a weight decay of 0.0001 was used, but the Adam optimizer was also tried due to its computational efficiency and suitability for sparse datasets.

The original network architecture included dropout layers between the linear perceptron layers, which yielded disastrous results. This is because dropout layers are designed to be useful for classification problems, which are less about demanding precision from every weight and more about determining which weights help with the classification task. In comparison, regression problems require the network to output a specific value and thus require each weight to be used and trained to utmost precision. It was found that batch normalization layers, which stabilized inputs for each batch, had a positive impact on model performance. ReLU (Rectified Linear Unit) layers were used throughout the architecture's development, but several other activation

layers were experimented with in attempts to find smoother activation functions. These included the SiLU (Sigmoid Linear Unit) activation function and the Mish activation function. It was also discovered from node tuning that using very few layers but keeping node counts large throughout the neural network optimized the performance of the neural network.

The Decision Tree Regressor was tested, with hyperparameters including the maximum depth of the tree and the volatility of node splitting within the tree, or how responsive it was to new information. This model yielded one of the best results yet, but with such a high sample count and feature count, it was necessary to test a Random Forest Regressor of many trees. The same two hyperparameters for the decision tree were tuned to the same values as the decision tree, but the tree count also had to be tuned to a value of 50 trees. This model yielded the best result of all independently developed models.

VI Final Prediction Model

To explore a pipeline as a possible final prediction model using all of the development information, the best transformers and models were considered. The variance threshold was the best transformer, and the optimal L1 model and optimal random forest were the best models. However, a pipeline can only have one model, the L1 model had already been tested on the variance threshold as part of model development. Thus, the only new option that was left was to try a pipeline with a variance threshold and a random forest. After tuning all hyperparameters, this model turned out to be even better than the random forest alone at the regression task. The pipeline was thus used as the final prediction model for the regression task. It was able to improve on the original MSE of the plain Linear Regression model by nearly ten percent.

VII Feature Analysis

More often than not, machine learning models are analogous to a black box. Aside from tuning hyperparameters using guesswork, training the model consists of feeding the model an input and analyzing the output. In problems involving high-dimensional data, pinpointing the impact of individual features or focusing on their relationships with each other are rarely considered. However, there are many different techniques that can be used in feature-centric analysis of a machine learning problem.

One of the most intuitive and common methods for analyzing the individual relationships of features to the target is the r statistic or correlation coefficient. An r value from 0 to 1 indicates a positive relationship, and an r value from -1 to 0 indicates a negative relationship. Values near 1 or -1 signify strong relationships, while an r value near 0 points to a weak or nonexistent relationship. Thus, r -regression was run on all of the features with respect to the testing time. The overall results are shown in Figure 1.

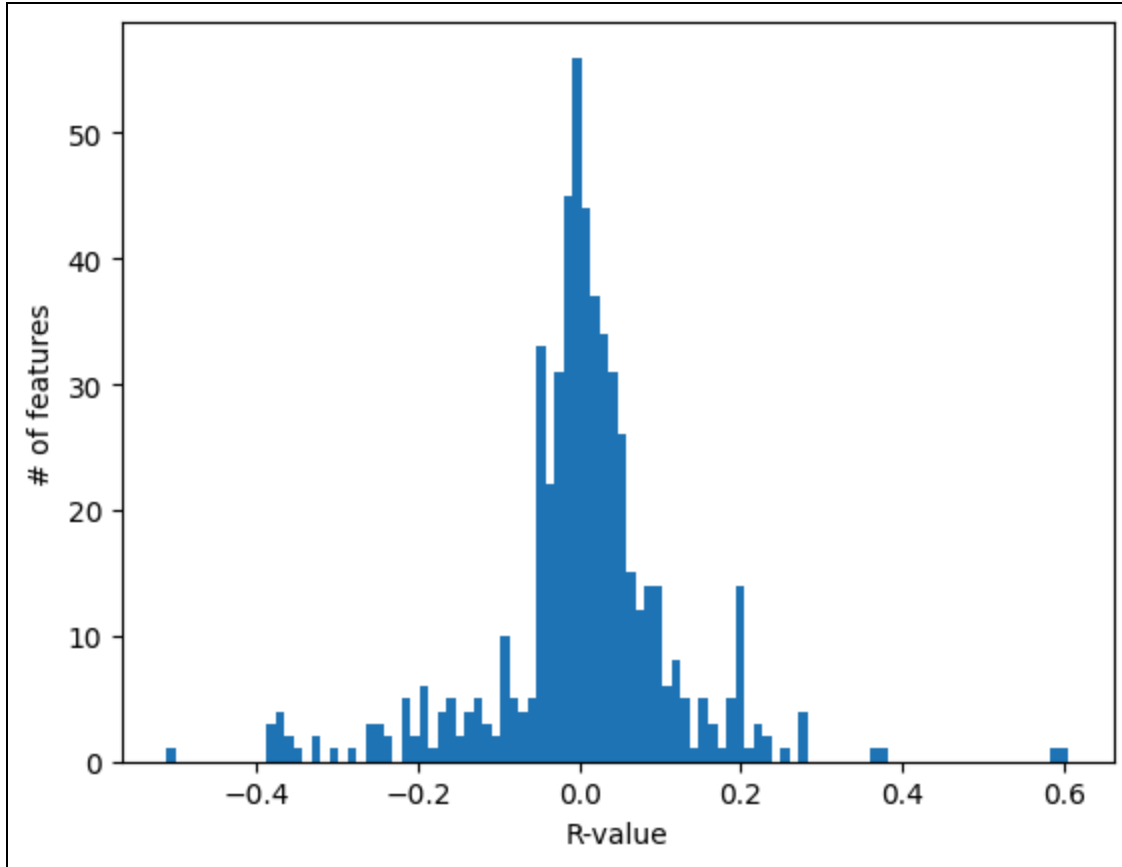


Figure 1: Histogram showing distribution of r-values across all features

The results were that three features had a somewhat strong correlation - X314 and X261 had a positive correlation while X127 had a negative correlation. About 60 percent of the features did not have a significant relationship to the target variable. Most variables with a weak or strong correlation (defined as r having a magnitude greater than 0.25) had a negative correlation, while most variables with any significant correlation (defined as r having a magnitude greater than 0.05) had a positive correlation.

Another important assumption that feature analysis is capable of addressing is the assumption that more variance makes a feature more important in the context of prediction. To test this

assumption, the variance of each feature was plotted against the importance of that feature. The list of feature importances was generated from fitting an optimal random forest model to the feature data and obtaining the feature importance attribute from the forest. The resulting graph is depicted in Figure 2.

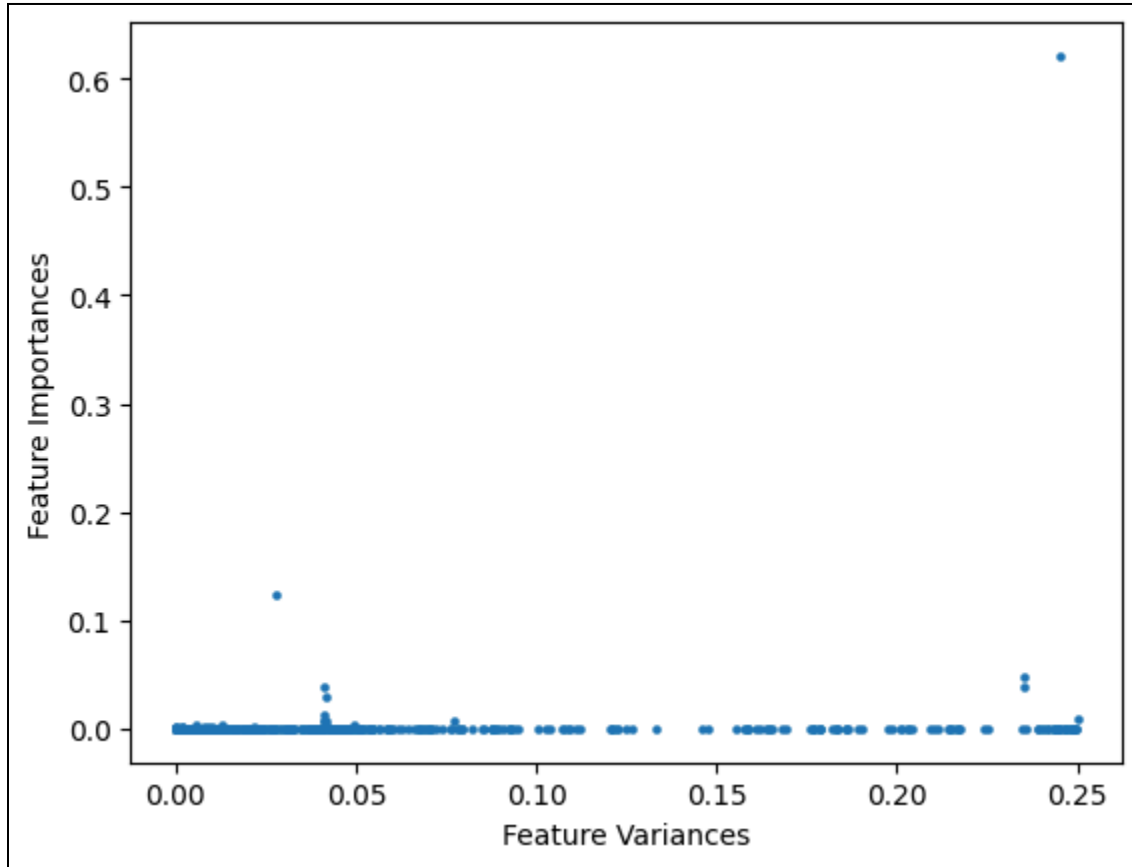


Figure 2: Scatter plot of feature variance versus feature importance within RF model

According to the graph, high-variance feature X314 has most of the importance within the random forest model, but low-variance feature X315 has the second-highest importance. However, the only high-variance features that had a significant importance (defined as being >0.001) were X118, X119, and X127. Interestingly, despite having the strongest statistical

correlation of the three, X127 was the least important of these three features. It can also be observed that there is a small band of eight features with significant importance and a variance of about 0.04. Thus, while the most important and potentially defining feature is a high-variance feature, there are many features with a low variance that do have a significant predictive power. The initial assumption about the relationship between variance and importance is not in accordance with this finding.

VIII Conclusions

This project consisted of two distinct parts that each yielded interesting conclusions about the task of predicting testing time for an automobile given its features. By considering results from both parts in tandem, several overarching conclusions can be drawn from this project.

It can be concluded that the MBGM dataset is a sparse dataset. This conclusion is supported by the success of the L1 regularization model because L1 penalties are meant to accommodate sparse datasets. This conclusion makes sense because roughly a third of the processed dataset's features were related to categorical data, and of all of the features corresponding to a certain category, only one can have a non-zero value for each sample. The one-hot encoding of categorical data promotes sparsity within the data from a machine-learning perspective.

It can be concluded that although the MBGM dataset has a high dimension, it has little collinearity within it. This conclusion is supported by the subpar results of models that are meant to account for collinearity and covariance, such as the L2 regularization model and the PLS regression model. This conclusion is significant because it was initially expected that many

features would share certain relationships, but this does not seem to be the case. This is confirmed by the results from the r-regression test, because most of the features have very low r-value magnitudes which leaves little room for collinearity.

It can be concluded that splitting the data into batches works best when the batches are used for different purposes. This conclusion is supported by comparing the worse than expected performance of stochastic models like the SGD regressor and the neural network to the great performance of the random forest model. While SGD-based models split up data into batches, these batches are all directly committed to the universal task of helping the model minimize a certain loss function. Random forest models use a technique known as “bagging” where different trees are given different sets of data, and thus each batch is put to the task of optimizing a different tree in the forest. This conclusion is significant because there may be more than one aspect that goes into using large datasets of cars to predict testing time.

It can be concluded that high variance does not necessarily imply a feature’s importance in the context of prediction. This conclusion is supported by the variance versus importance graph, in which it was observed that there was a group of low-variance features with significant importance within the random forest model. This conclusion is supported by the variance-based data transformations, in which the transformer that dropped only the least variant features outperformed the transformer that kept only the most variant features. The k-best features transformer effectively missed out on important low-variance features that the variance threshold transformer captured. This conclusion is significant because there may be features that are almost

always present in a car or almost always absent, but carry a significant impact on testing time. This includes categorical data with many categories, because each category is likely rare.

IX Discussion

There are several limitations to the procedures in this project. One is that while the model analysis included training sets and validation sets, although cross-validation was used to fit the model multiple times, there was no test set included. Although a similar dataset could not be found, this problem could have been avoided by using a train-test data split. In addition, it is hard to determine any unforeseen implications of the categorical data without actually knowing what the features or the categories are. In preprocessing, it was assumed that one-hot encoding the categories was all that was necessary, even though this assumption may not be fully valid. All of these limitations could be improved upon in the future of the project. However, certain limitations cannot as easily be lifted in the future. For instance, because the dataset is unlabeled, real-world inferences about actual car features are prevented.

X References

- [1] <https://www.openml.org/search?type=data&status=active&id=42570>
- [2] https://scikit-learn.org/stable/user_guide.html
- [3] <https://docs.pytorch.org/docs/stable/pytorch-api.html>